

arc42 – Softwaredokumentation

Dein großer Preis

Version 1.0 | 13. April 2026

Kurs: Software Engineering – TINF24B4

Team: Finn Berners (Product Owner), Johanna Rieschl (Scrum Master), Lona, Ferheng,
Denis

Repository: <https://github.com/Finn-B-DHBW/Der-Grosse-Preis>

1. Einführung und Ziele

1.1 Aufgabenstellung

"Dein großer Preis" ist eine Desktop-Applikation für das Veranstellen von Quiz-Abenden. Die Software ermöglicht es, eigene Kategorien und Fragen zu erstellen sowie fertige Spiele zu laden und zu spielen. Mehrere Teams können gegeneinander antreten; Punkte werden live verwaltet. Gespielt wird, bis alle Fragen ausgewählt wurden – das Team mit den meisten Punkten gewinnt.

1.2 Qualitätsziele

Priorität	Qualitätsziel	Szenario
1	Benutzbarkeit	Ein Spielleiter kann ein neues Spiel ohne Einarbeitungszeit starten.
2	Funktionale Eignung	Das System verwaltet Fragen, Kategorien und Spielstände korrekt und vollständig.
3	Änderbarkeit	Neue Fragekategorien oder Spielmodi können mit geringem Aufwand ergänzt werden.
4	Portabilität	Die Anwendung läuft plattformübergreifend auf Windows, macOS und Linux ohne Installation.
5	Zuverlässigkeit	Spielstände und Fragen gehen bei einem Absturz nicht verloren.

1.3 Stakeholder

Rolle	Erwartung
Spielleiter	Schnelles Aufsetzen und reibungsloses Steuern des Quiz
Spieler / Teams	Faire Punktevergabe, übersichtliche Anzeige
Entwicklungsteam	Klar strukturierter, wartbarer Code
Dozent (DHBW)	Dokumentierte, nachvollziehbare Architekturentscheidungen

2. Randbedingungen

2.1 Technische Randbedingungen

Randbedingung	Beschreibung
Programmiersprache	Java 25
Build-Tool	Apache Maven
GUI-Framework	Java Swing mit FlatLaf 3.4 (modernes Look & Feel)
Datenbank	SQLite via sqlite-jdbc 3.45.3.0 – lokale Datei database.db
Deployment	Ausführbares Fat-JAR (erzeugt durch maven-shade-plugin)
Plattform	Windows, macOS, Linux (überall wo JVM vorhanden)
Versionsverwaltung	Git / GitHub

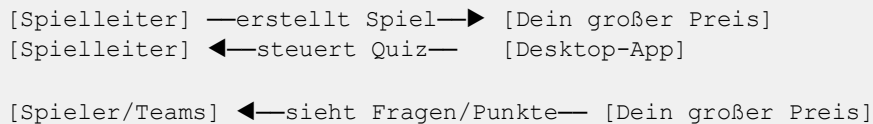
2.2 Organisatorische Randbedingungen

- Agile Entwicklung mit Scrum (Jira, 1-Wochen-Sprints, Sprint-Meeting donnerstags 12:30 Uhr)
- Daily montags 13:45–14:00 Uhr
- Keine externe Server-Infrastruktur erlaubt (reine Offline-Desktop-App)

3. Kontextabgrenzung

3.1 Fachlicher Kontext

Systemkontext



Das System interagiert ausschließlich mit menschlichen Akteuren. Es gibt keine externen Systeme, APIs oder Netzwerkdienste.

3.2 Technischer Kontext



4. Lösungsstrategie

Die Architektur folgt einem klassischen geschichteten Ansatz (Layered Architecture) für eine Desktop-Applikation:

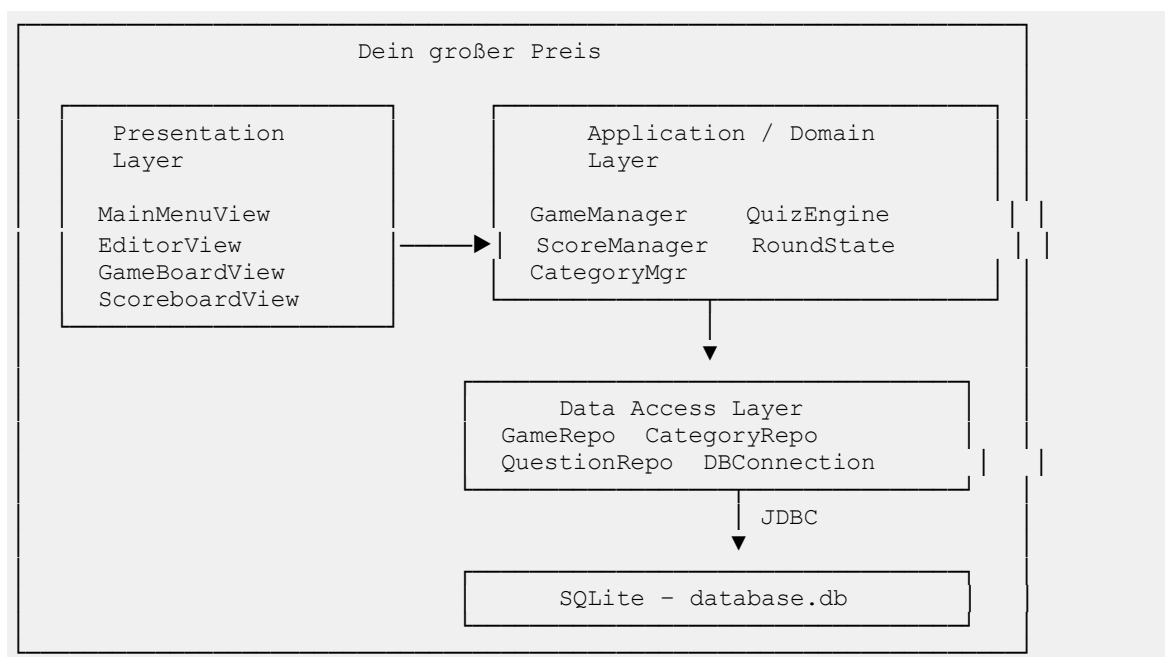
- **Presentation Layer** (Swing + FlatLaf): Verwaltet alle UI-Screens (Hauptmenü, Fragen-Editor, Spielfeld, Scoreboard).

- **Application / Domain Layer:** Enthält die Spiellogik (Rundenverwaltung, Punkteberechnung, Zustandsmaschine des Spiels).
- **Data Access Layer:** Kapselt alle SQLite-Zugriffe über JDBC; stellt Repositories für Spiele, Kategorien und Fragen bereit.
- **Persistenz:** Lokale SQLite-Datei database.db im Arbeitsverzeichnis.

Diese Aufteilung trennt Darstellung, Logik und Datenhaltung sauber voneinander und ermöglicht unabhängige Änderbarkeit der Schichten.

5. Bausteinsicht

5.1 Ebene 1 – Gesamtsystem (Komponentendiagramm)



5.2 Paketstruktur (Ebene 2)

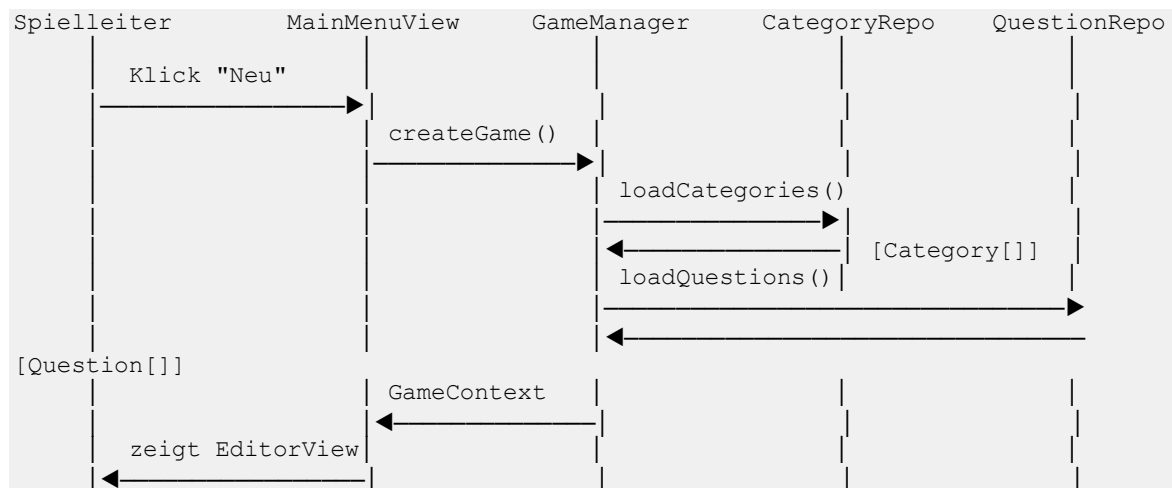
src/main/java/	
├ Main.java	← Einstiegspunkt, startet die Applikation
├ ui/	
│ └ MainMenuView.java	← Hauptmenü (Neues Spiel, Laden, Beenden)
│ └ EditorView.java	← Fragen- und Kategorien-Editor
│ └ GameBoardView.java	← Spielfeld (Kategorien × Schwierigkeitsstufen)
│ └ ScoreboardView.java	← Live-Punkte-Anzeige
├ game/	
│ └ GameManager.java	← Orchestriert einen Spieldurchlauf
│ └ QuizEngine.java	← Bestimmt Fragen-Reihenfolge, prüft Antworten
│ └ ScoreManager.java	← Hält Teampunkte, berechnet Gewinner
│ └ RoundState.java	← Zustandsmaschine: WAITING → QUESTION → ANSWER → DONE
├ CategoryManager.java	← Verwaltung von Kategorien im laufenden Spiel
├ data/	
│ └ DBConnection.java	← SQLite-Verbindungs-Singleton
│ └ GameRepository.java	← CRUD für gespeicherte Spiele

CategoryRepository.java	← CRUD für Kategorien
QuestionRepository.java	← CRUD für Fragen und Antwortoptionen

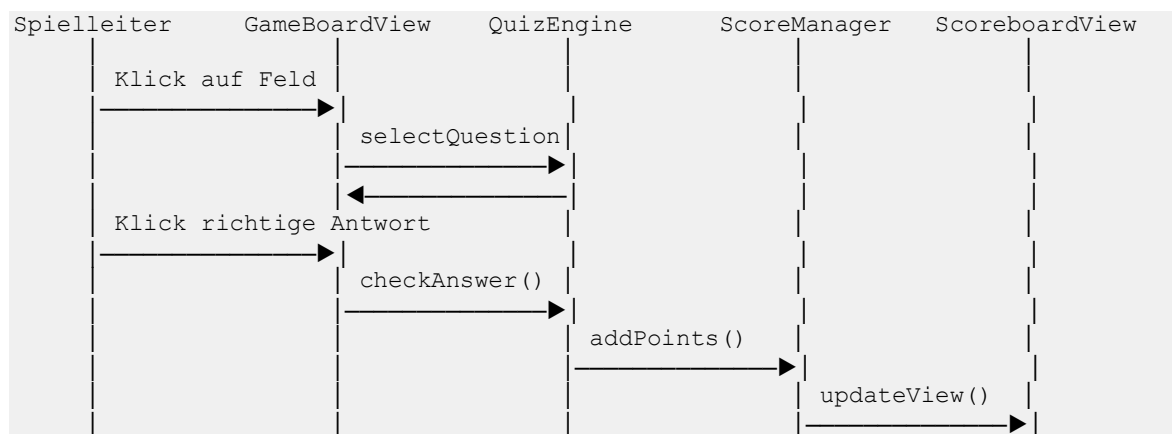
Hinweis: Die Paketstruktur basiert auf der bekannten Systemstruktur und dem im Repository sichtbaren Code; exakte Klassennamen können leicht abweichen und sind im Laufe des Semesters zu verifizieren.

6. Laufzeitsicht

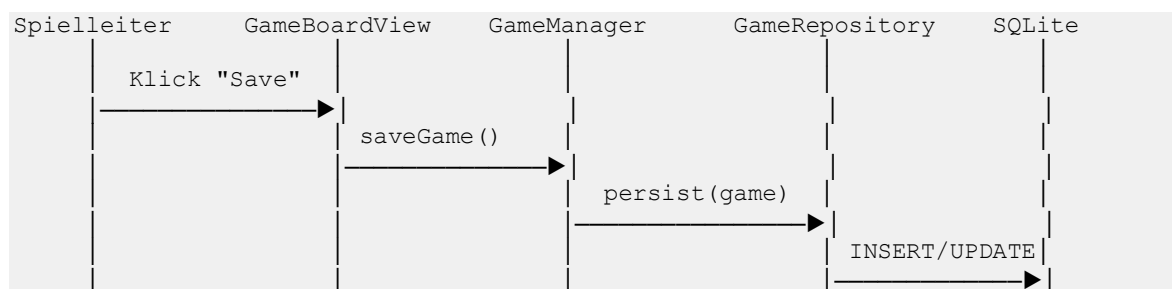
6.1 Szenario: Neues Spiel erstellen und starten



6.2 Szenario: Team wählt Frage, Antwort wird bewertet



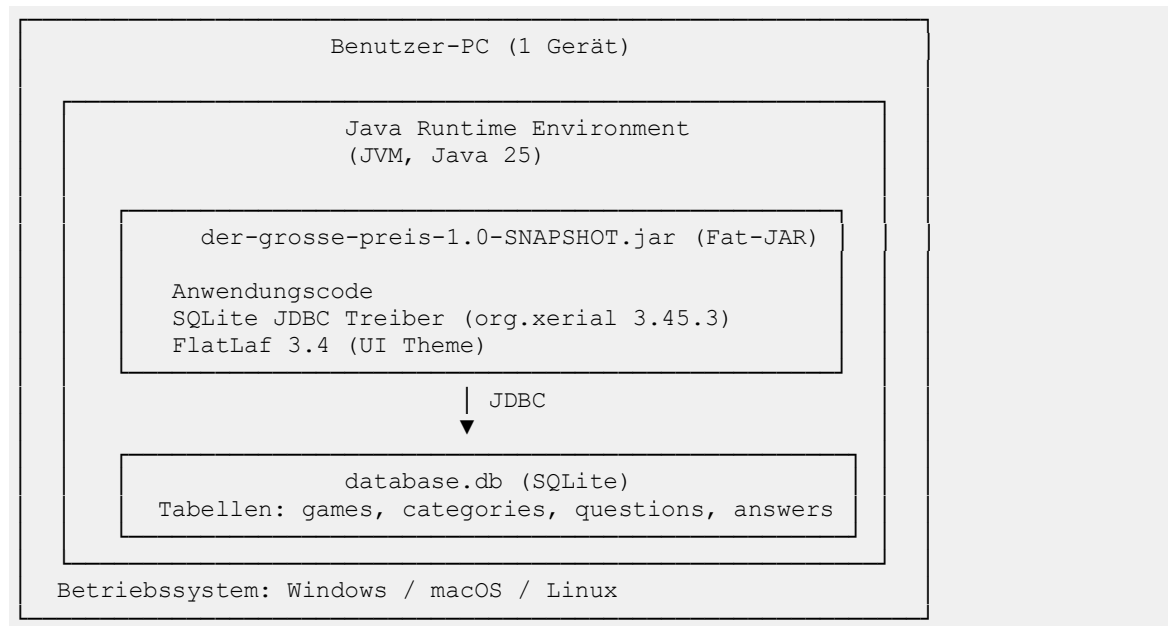
6.3 Szenario: Spiel speichern





7. Verteilungssicht

7.1 Verteilungsdiagramm



Die Anwendung läuft vollständig lokal auf einem einzigen Gerät. Es gibt keine Netzwerkkommunikation und keine Client-Server-Trennung. Die Datenbank-Datei `database.db` liegt im gleichen Verzeichnis wie die JAR-Datei. Das Fat-JAR enthält alle Abhängigkeiten (SQLite-JDBC, FlatLaf) und benötigt ausschließlich eine Java-Runtime (JVM 25).

8. Querschnittliche Konzepte

8.1 Persistenz

Alle persistenten Daten werden in einer lokalen SQLite-Datenbankdatei (`database.db`) gespeichert. Der Zugriff erfolgt ausschließlich über das `DBConnection-Singleton`, das die JDBC-Verbindung verwaltet. Repository-Klassen kapseln alle SQL-Statements; der Rest der Applikation hat keinen direkten DB-Zugriff.

Datenbankschema (vereinfacht):

```
CREATE TABLE games (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE categories (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    game_id INTEGER REFERENCES games(id),  
    name TEXT NOT NULL  
);
```

```

CREATE TABLE questions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    category_id INTEGER REFERENCES categories(id),
    text TEXT NOT NULL,
    points INTEGER NOT NULL,
    answered BOOLEAN DEFAULT FALSE
);

CREATE TABLE answers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    question_id INTEGER REFERENCES questions(id),
    text TEXT NOT NULL,
    is_correct BOOLEAN NOT NULL
);

```

8.2 UI-Gestaltung und Look & Feel

FlatLaf stellt ein modernes, plattformunabhängiges Look & Feel für Swing bereit. Es wird beim Start der Applikation in Main.java initialisiert. Alle Swing-Komponenten profitieren automatisch vom FlatLaf-Theme, ohne dass in einzelnen Views manuelle Styling-Anpassungen notwendig sind.

8.3 Fehlerbehandlung

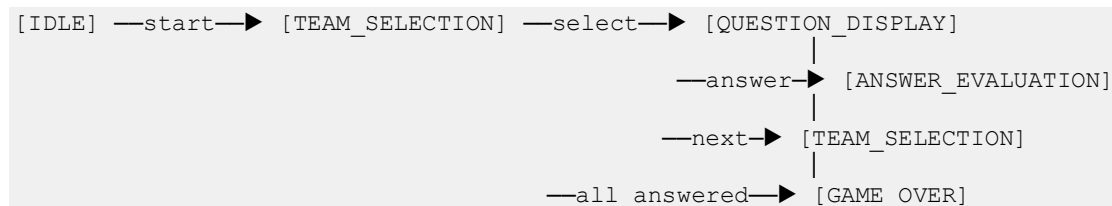
Datenbankfehler (z. B. SQLException) werden an der Datenzugriffsschicht abgefangen und als anwendungsspezifische Exceptions weitergegeben. Die UI zeigt dem Nutzer verständliche Fehlermeldungen per JOptionPane. Die Anwendung bricht nicht unkontrolliert ab.

8.4 Build und Deployment

Maven übernimmt das Build-Management. Das maven-shade-plugin erzeugt ein ausführbares Fat-JAR, das alle Abhängigkeiten enthält. Damit entfällt eine separate Installation von Bibliotheken; Nutzer benötigen nur eine JVM (Java 25).

8.5 Spielzustand (Zustandsmaschine)

Der Spielablauf folgt einer klar definierten Zustandsmaschine:



9. Architekturentscheidungen

ADR-01: Java Swing als UI-Framework

Kontext: Auswahl eines GUI-Frameworks für eine Desktop-Applikation in Java.

Entscheidung: Java Swing (ergänzt durch FlatLaf für modernes Styling).

Begründung: Das Team verfügt über Java-Kenntnisse aus dem Studium. Swing ist im JDK enthalten, erfordert keine zusätzlichen Laufzeitabhängigkeiten und ermöglicht

plattformübergreifende Desktop-Apps. FlatLaf löst das veraltete Standard-Look-and-Feel von Swing.

Alternativen verworfen: JavaFX (höhere Lernkurve, separates SDK), Web-Technologien (Overhead für eine lokale Desktop-App).

ADR-02: SQLite als Datenbank

Kontext: Persistenz von Spielen, Kategorien und Fragen.

Entscheidung: SQLite via sqlite-jdbc.

Begründung: SQLite benötigt keinen separaten Datenbankserver, ist dateibasiert und damit ideal für eine Einzelplatz-Desktop-App. Der SQLite-JDBC-Treiber lässt sich problemlos ins Fat-JAR einbinden.

Alternativen verworfen: H2 (ähnlich, aber weniger verbreitet), JSON-Dateien (kein relationales Modell, schlechte Abfragemöglichkeiten), Externe DB-Server (unnötige Infrastruktur).

ADR-03: Fat-JAR als Deployment-Artefakt

Kontext: Verteilung der Anwendung an Endnutzer ohne Installationsroutine.

Entscheidung: Erzeugung eines Fat-JARs über maven-shade-plugin.

Begründung: Ein einziges JAR-File vereinfacht die Weitergabe erheblich. Nutzer benötigen nur eine JVM und können die Anwendung per Doppelklick oder `java -jar` starten.

ADR-04: Lokale Einzelplatz-Architektur (kein Client-Server)

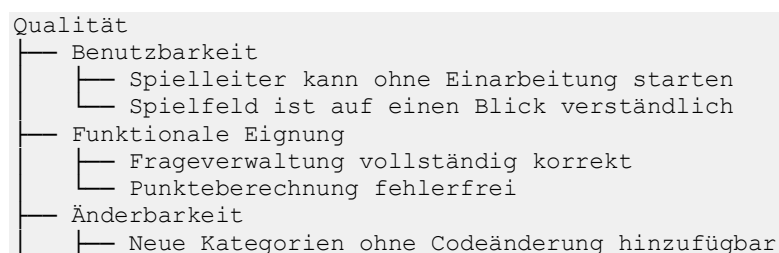
Kontext: Mehrere Teams spielen gleichzeitig am gleichen Gerät (ein Bildschirm, ein Spielleiter).

Entscheidung: Keine Netzwerkarchitektur; ein Prozess, ein Gerät, ein Spielleiter steuert den Ablauf.

Begründung: Das Anwendungsszenario ist ein gemeinsamer Quiz-Abend an einem Rechner/Bildschirm. Netzwerkfähigkeit würde die Komplexität deutlich erhöhen, ohne konkreten Mehrwert für das Kernszenario.

10. Qualitätsanforderungen

10.1 Qualitätsbaum



└─	Neue Spielmodi mit geringem Aufwand integrierbar
└─	Portabilität
└─	Läuft auf Windows, macOS, Linux
└─	Keine Installation außer JVM nötig
└─	Zuverlässigkeit
└─	Daten gehen bei Absturz nicht verloren
└─	SQLite-Transaktionen sichern Konsistenz

10.2 Qualitätsszenarien

ID	Qualitätsziel	Szenario	Reaktion	Messbar
QS-1	Benutzbarkeit	Spielleiter startet erstmalig die App	Hauptmenü erscheint mit Buttons	Ohne Dokumentation in 2 Min. spielbereit
QS-2	Funktionale Eignung	Team gibt richtige Antwort	Punkte korrekt addiert, Feld markiert	0 Fehler in 100 Testrunden
QS-3	Änderbarkeit	Entwickler fügt neue Schwierigkeitsstufe hinzu	Nur Data-Layer und Editor-View betroffen	≤ 2 Klassen zu ändern
QS-4	Portabilität	App wird auf macOS gestartet	Identische GUI wie auf Windows	Manuelle Tests auf allen Plattformen bestanden
QS-5	Zuverlässigkeit	App stürzt während Spielrunde ab	Letzter Zustand wiederherstellbar	Kein Datenverlust nach Auto-Save

11. Risiken und technische Schulden

11.1 Risiken

11. Risiken und technische Schulden

Inhalt

Dieser Abschnitt beschreibt die wichtigsten identifizierten Risiken des Projekts „Der große Preis“. Berücksichtigt werden sowohl projektspezifische Risiken als auch solche, die sich aus der gewählten Architektur, dem Technologie-Stack sowie den organisatorischen Rahmenbedingungen ergeben.

Eine detaillierte Auflistung aller identifizierten Risiken inklusive Bewertung (Wahrscheinlichkeit, Schadenshöhe), Indikatoren sowie konkreter Maßnahmen ist in der separaten RMMM-Tabelle dokumentiert (siehe [Der-Grosse-Preis/Projektdateien/Risikomanagement/RMMM_Liste.pdf at main · Finn-B-DHBW/Der-Grosse-Preis](#)).

Für das vorliegende System sind insbesondere solche Risiken relevant, die

- die rechtzeitige Fertigstellung eines lauffähigen Systems gefährden,
- die Erreichung zentraler Qualitätsziele (z. B. Portabilität und Zuverlässigkeit) beeinträchtigen,
- die korrekte Verarbeitung von Spielzuständen, Punkten und Persistenzdaten gefährden,

- die Wartbarkeit und Erweiterbarkeit der Architektur einschränken, oder
- durch Zeitdruck zur Entstehung technischer Schulden führen.

Risikomanagement

Motivation

Das Projekt wird im Rahmen eines studentischen Software-Engineering-Kurses entwickelt. Daraus ergeben sich typische Risiken wie eine begrenzte Entwicklungszeit innerhalb des Semesters, parallele Belastung durch andere Lehrveranstaltungen sowie ein heterogener Erfahrungsstand im Team.

Zusätzlich ergeben sich architekturelevante Risiken aus den gewählten technischen Entscheidungen. Dazu zählen insbesondere die plattformübergreifende Ausführung einer Java-Swing-Anwendung, die Nutzung einer lokalen SQLite-Datenbank sowie die Umsetzung einer klar getrennten Schichtenarchitektur. Fehler oder Einschränkungen in diesen Bereichen können direkte Auswirkungen auf zentrale Qualitätsziele wie Portabilität, Zuverlässigkeit und Änderbarkeit haben.

Ein weiterer kritischer Faktor ist der Umgang mit Persistenzdaten. Änderungen am Datenbankschema oder fehlerhafte Speicherung von Spielzuständen können zu Inkonsistenzen oder Datenverlust führen und damit das Vertrauen in die Anwendung beeinträchtigen.

Neben klassischen Projektrisiken spielt auch das Thema technische Schulden eine wichtige Rolle. Diese entstehen insbesondere dann, wenn unter Zeitdruck pragmatische Lösungen implementiert werden, die kurzfristig funktional sind, jedoch langfristig Wartbarkeit, Verständlichkeit und Erweiterbarkeit der Codebasis verschlechtern.

Typische Beispiele im vorliegenden Projekt sind mögliche Vereinfachungen im Datenmodell, eingeschränkte Fehlerbehandlung oder fehlende Migrationsmechanismen für Persistenzdaten.

Form

Die systematische Erfassung, Bewertung und Behandlung der Risiken erfolgt über eine RMMM-Tabelle (Risk Mitigation, Monitoring, Management). Diese enthält für jedes identifizierte Risiko:

- eine Beschreibung,
- eine quantitative Bewertung (Wahrscheinlichkeit und Schadenshöhe),
- Maßnahmen zur Risikominimierung,
- Indikatoren zur frühzeitigen Erkennung sowie
- konkrete Notfallstrategien.

Die Tabelle dient als zentrales Instrument zur kontinuierlichen Risikoüberwachung im Projektverlauf und wird regelmäßig aktualisiert.

12. Glossar

Begriff	Erklärung
Fat-JAR	JAR-Datei, die alle Abhängigkeiten enthält und direkt ausführbar ist
FlatLaf	Modernes Look-and-Feel-Framework für Java Swing
SQLite	Dateibasiertes relationales Datenbanksystem ohne separaten Server
Spielleiter	Person, die die Quiz-Software bedient und den Spielablauf steuert

Begriff	Erklärung
Kategorie	Themengebiet, dem Fragen zugeordnet sind (z. B. "Geschichte", "Sport")
Schwierigkeitsstufe	Punktwert einer Frage (z. B. 100, 200, 300, 400, 500 Punkte)
GameBoard	Spielfeld mit Kategorien und Schwierigkeitsstufen wie bei "Jeopardy"
ADR	Architecture Decision Record – dokumentiert eine Architekturentscheidung
arc42	Bewährtes Template zur Dokumentation von Softwarearchitekturen